



OpenSplice DDS

## **Evaluation and Benchmarking Guide**

*Release 7.x*

# CONTENTS

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| <b>1</b>  | <b>Preface</b>                                           | <b>1</b>  |
| 1.1       | About the Evaluation & Benchmarking Guide . . . . .      | 1         |
| 1.2       | Conventions . . . . .                                    | 1         |
| <b>2</b>  | <b>Introduction</b>                                      | <b>2</b>  |
| <b>3</b>  | <b>OpenSplice Basics</b>                                 | <b>3</b>  |
| <b>4</b>  | <b>OpenSplice Architectural Modes</b>                    | <b>5</b>  |
| 4.1       | The Single Process or Standalone deployment . . . . .    | 5         |
| 4.2       | The Shared Memory or Federated deployment . . . . .      | 6         |
| 4.3       | How to select the Architectural Mode . . . . .           | 6         |
| <b>5</b>  | <b>OpenSplice Networking Options</b>                     | <b>8</b>  |
| 5.1       | How to select the Networking Protocol . . . . .          | 8         |
| <b>6</b>  | <b>Benchmarking OpenSplice: Decision Trees</b>           | <b>9</b>  |
| <b>7</b>  | <b>How to run OpenSplice</b>                             | <b>13</b> |
| 7.1       | The OpenSplice Environment . . . . .                     | 13        |
| 7.2       | Running Single Process and Shared Memory Modes . . . . . | 13        |
| <b>8</b>  | <b>Performance Tests and Examples</b>                    | <b>14</b> |
| 8.1       | Round-Trip Latency Performance . . . . .                 | 14        |
| 8.2       | Throughput Performance . . . . .                         | 15        |
| <b>9</b>  | <b>Bibliography</b>                                      | <b>18</b> |
| <b>10</b> | <b>Contacts &amp; Notices</b>                            | <b>19</b> |
| 10.1      | Contacts . . . . .                                       | 19        |
| 10.2      | Notices . . . . .                                        | 19        |

## 1.1 About the Evaluation & Benchmarking Guide

The *Evaluation & Benchmarking Guide* is the starting point for anyone who wants to evaluate OpenSplice.

*This Evaluation Guide contains:*

- a general introduction to ...
- descriptions of how to ...
- *etc.*

## 1.2 Conventions

The icons shown below are used in ZettaScale product documentation to help readers to quickly identify information relevant to their specific use of OpenSplice.

| <i>Icon</i>                                                                         | <i>Meaning</i>                                                          |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
|  | Item of special significance or where caution needs to be taken.        |
|  | Item contains helpful hint or special information.                      |
|  | Information applies to Windows ( <i>e.g.</i> XP, 2003, Windows 7) only. |
|  | Information applies to Unix-based systems ( <i>e.g.</i> Solaris) only.  |
|  | Information applies to Linux-based systems ( <i>e.g.</i> Ubuntu) only.  |
|  | C language specific.                                                    |
|  | C++ language specific.                                                  |
|  | C# language specific.                                                   |
|  | Java language specific.                                                 |

## INTRODUCTION

One of the key differentiators of OpenSplice is that it provides a user with the ability to choose exactly how to deploy Data Distribution Service (DDS) applications, *i.e.* there are different DDS system architecture deployment modes and also different networking service protocols. This allows a user to maximize both *intra*-nodal and *inter*-nodal performance based on requirements specific to their own use case. When evaluating OpenSplice it is very important to understand all of these features and benefits to ensure that the most appropriate combination is evaluated against your specific performance criteria. Once the performance figures have been observed the choice is usually clear.

Every customer use case and set of requirements is different, so let us briefly guide you through how to best deploy OpenSplice so that it meets and exceeds your expectations. Here we explain how easy it is to get started with OpenSplice and observe the excellent performance and scalability it provides. OpenSplice is even shipped with dedicated performance tests that the user can build and run easily.

*Note that this Guide serves only as an introduction and does not replace the full OpenSplice reference and user guides.*

## OPENSPLICE BASICS

OpenSplice is configured using an XML configuration file. In this file, the user specifies the architectural model and the OpenSplice services that are to run when the DDS infrastructure is started.

The `OSPL_URI` environment variable refers to the specific XML configuration file that is used for the current deployment. The default value refers to the `ospl.xml` file located in the `etc/config` directory of the OpenSplice installation. The installation directory itself can be referred to by the `OSPL_HOME` environment variable. Please see [The OpenSplice Environment](#) for details of how to set up the OpenSplice environment.

A number of other sample configuration files that can be used when benchmarking OpenSplice are also provided in the `etc/config` directory.

The `OSPL_URI` variable is of the form:

**Linux** `OSPL_URI=file://$OSPL_HOME/etc/config/ospl.xml`

**Windows** `OSPL_URI=file://%OSPL_HOME%\etc\config\ospl.xml`

You can refer to the `OpenSplice_DeploymentGuide.pdf` later for more details of the `OSPL_URI` variable; for now, let us see what aspects of the OpenSplice deployment are controlled by this file.

**i** The OpenSplice Launcher tool assists with the selection of the `OSPL_URI` variable. There is a *Configurations* menu that lists the sample configuration files that are available.



Fig. 1: The Launcher tool

The OpenSplice Launcher tool is also able to run the examples and performance tests that are described later in this document.

## OPENSPLICE ARCHITECTURAL MODES

OpenSplice provides two main architectural modes. These are the **Single Process** deployment mode, which provides a **Standalone** architecture, and, unique to OpenSplice, the **Shared Memory** deployment mode which provides a **Federated** architecture.

### 4.1 The Single Process or Standalone deployment

Features of this mode are:

- Simplest to run and get started with.
- Each DDS application process contains the entire DDS infrastructure.
- Uses in-process heap memory for the DDS database.
- OpenSplice services run as threads within the application process.
- When there are multiple DDS application processes on a single machine, the communication between them must be done *via* a networking service; this introduces additional overhead so performance in this scenario is not optimal.

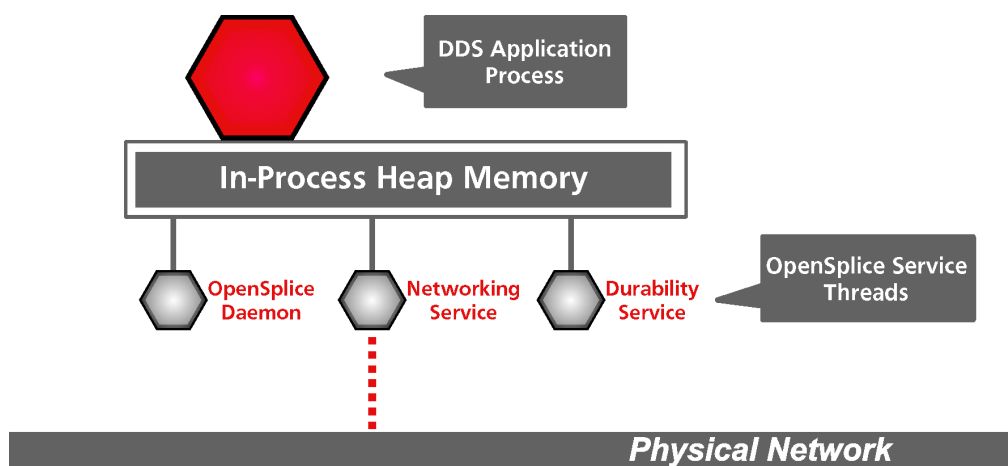


Fig. 1: Single Process or Standalone deployment

## 4.2 The Shared Memory or Federated deployment

Features of this mode are:

- The DDS infrastructure is started once per machine.
- Uses shared memory for the DDS database.
- Each DDS application process interfaces with the shared memory rather than creating the DDS infrastructure itself.
- Allows the data to be physically present only once on any machine.
- Reading and writing directly to locally-mapped memory is far more efficient than having to actually move the data *via* a networking service, allowing for improved performance and scalability.
- OpenSplice services are able to arbitrate over all of the DDS data on the node, and so can make smart decisions with respect to data delivery so that priority QoS values (for example) are respected; this is not possible when there are multiple standalone deployments on a machine.

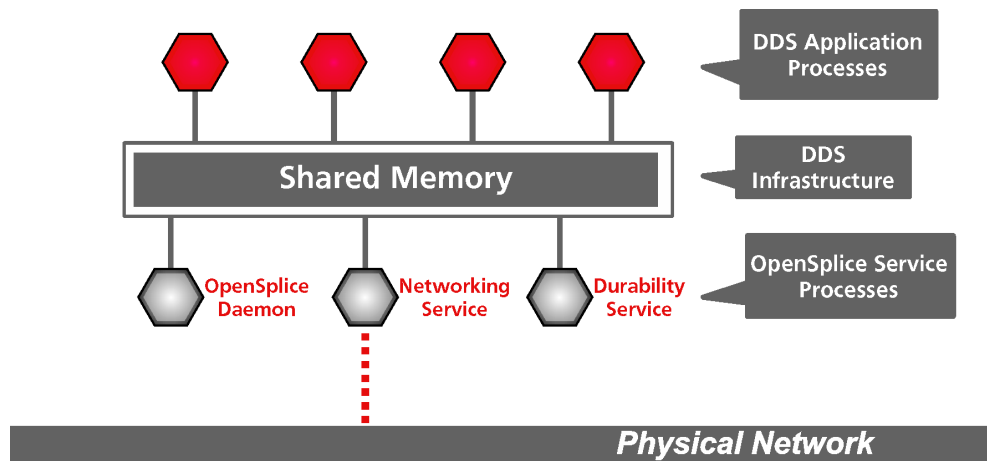


Fig. 2: Shared Memory or Federated deployment

When there are multiple DDS applications running on a single computing node, the use of OpenSplice's unique Shared Memory architecture can provide greater performance, smaller footprint and better scalability than other DDS deployment options.

## 4.3 How to select the Architectural Mode

- For a Single Process deployment, set the OSPL\_URI variable to refer to a single process (sp) xml file such as `ospl_sp_dds.xml` or `ospl_sp_nativeRT.xml`. Note that a networking service (such as `dds` or `nativeRT`) is required for two DDS application processes to communicate even if they are running on the same physical machine. See the next section for more details on networking options.

A single process deployment is enabled when the Domain section of the XML configuration contains a '`<SingleProcess> TRUE`' attribute.

NOTE for VxWorks kernel mode builds of OpenSplice the single process feature of the OpenSplice domain must not be enabled. i.e. "`<SingleProcess>true</SingleProcess>`" must not be included in the OpenSplice Configuration xml. The model used on VxWorks kernel builds is always that an area of kernel memory is allocated to store the domain database ( the size of which is controlled by the size option in the Database configuration for opensplice as is used on other platforms for the shared memory model. ) This can then be accessed by any task on the same VxWorks node.



- For a Shared Memory deployment, set the OSPL\_URI variable to refer to a shared memory (*shmem*) xml file such as `ospl_shmem_no_network.xml`, `ospl_shmem_dds.xml`, or `ospl_shmem_nativeRT.xml`. Note that two or more DDS applications running on the same physical machine are able to communicate *via* the shared memory so a networking service (such as `dds` or `nativeRT`) is not necessarily required.

**A shared memory deployment is enabled when the Domain section of the XML configuration does not contain a '`<SingleProcess> TRUE`' attribute but does contain a '`<Database>`' attribute.**



Note that by default the OSPL\_URI environment variable refers to a *Single Process* configuration, so to see the extra performance and scalability benefits of OpenSplice's Shared Memory architecture it is necessary to switch from the default.

## OPENSPLICE NETWORKING OPTIONS

OpenSplice provides several networking options for the delivery of DDS data between nodes. The networking service selection is largely transparent to the user; the difference is observed in the CPU consumption, networking load, and ultimately how fast and efficiently the data is delivered between nodes. The most applicable service is dependent on the requirements of the use case.

**OpenSplice DDSI** is the industry standard protocol providing vendor interoperability that operates using a typed ‘pull’ style model. It additionally provides features, such as channels, partitioning and encryption, which are not in the standard.

**OpenSplice RTNetworking** is an alternative to the DDSI wire protocol. RTNetworking uses a type-less ‘push’ style model in contrast to DDSI and is often the more performant, scalable option. RTNetworking also offers prioritization of network traffic via ‘channels’, partitioning to separate data flows and optional compression for low-bandwidth environments. **OpenSplice SecureRTNetworking** provides these features together with encryption and access control.

### 5.1 How to select the Networking Protocol

As with the architectural deployment choice, the selection of the networking service is described by the XML configuration file. Note that this choice is independent of and orthogonal to the architectural deployment: you can have single process or shared memory with any of the networking service protocols.

- To run with a **DDSI** service, set the `OSPL_URI` variable to refer to a DDSI xml file such as `ospl_sp_dds.xml` or `ospl_shmem_dds.xml`.
- To run with an **RTNetworking** service, set the `OSPL_URI` variable to refer to an RTNetworking xml file such as `ospl_sp_nativeRT.xml` or `ospl_shmem_nativeRT.xml`.
- To run with a **SecureRTNetworking** service, set the `OSPL_URI` variable to refer to the `ospl_shmem_secure_nativeRT.xml` SecureRTNetworking xml file.



Note that by default, the `OSPL_URI` environment variable refers to a *DDSI* configuration, so to see the extra performance and scalability benefits of OpenSplice’s RTNetworking it is necessary to switch from the default.

## **BENCHMARKING OPENSPLICE: DECISION TREES**

DDS provides many functional benefits that set it apart from other middleware technologies, but users often still have specific performance requirements for latency, throughput, CPU and network utilization. OpenSplice provides the functional benefits of the technology whilst remaining committed to excellent performance.

The flowcharts in this chapter show the decision criteria that may be applied in order to decide on the most appropriate test case, architectural mode, and networking protocol options for your specific use case and requirements.

. How to run OpenSplice:

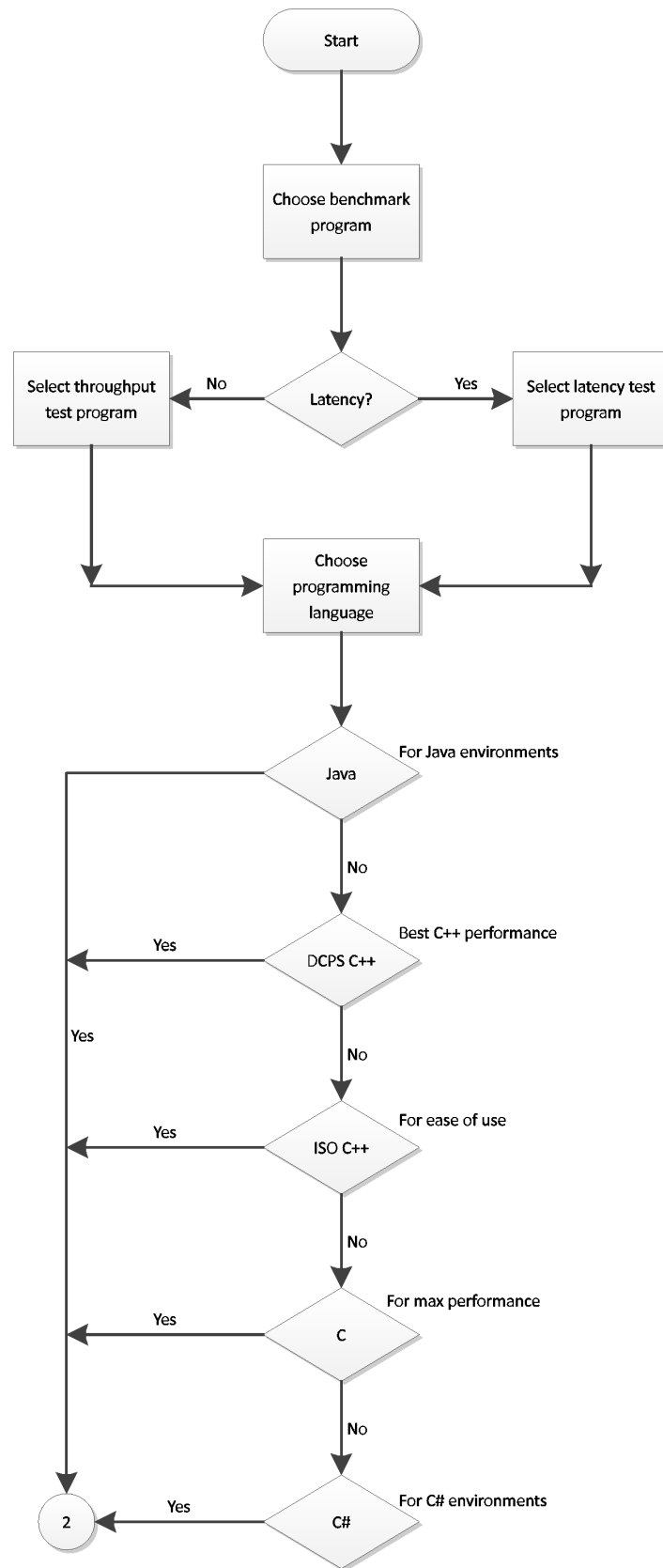


Fig. 1: Selecting a specific performance test and programming language

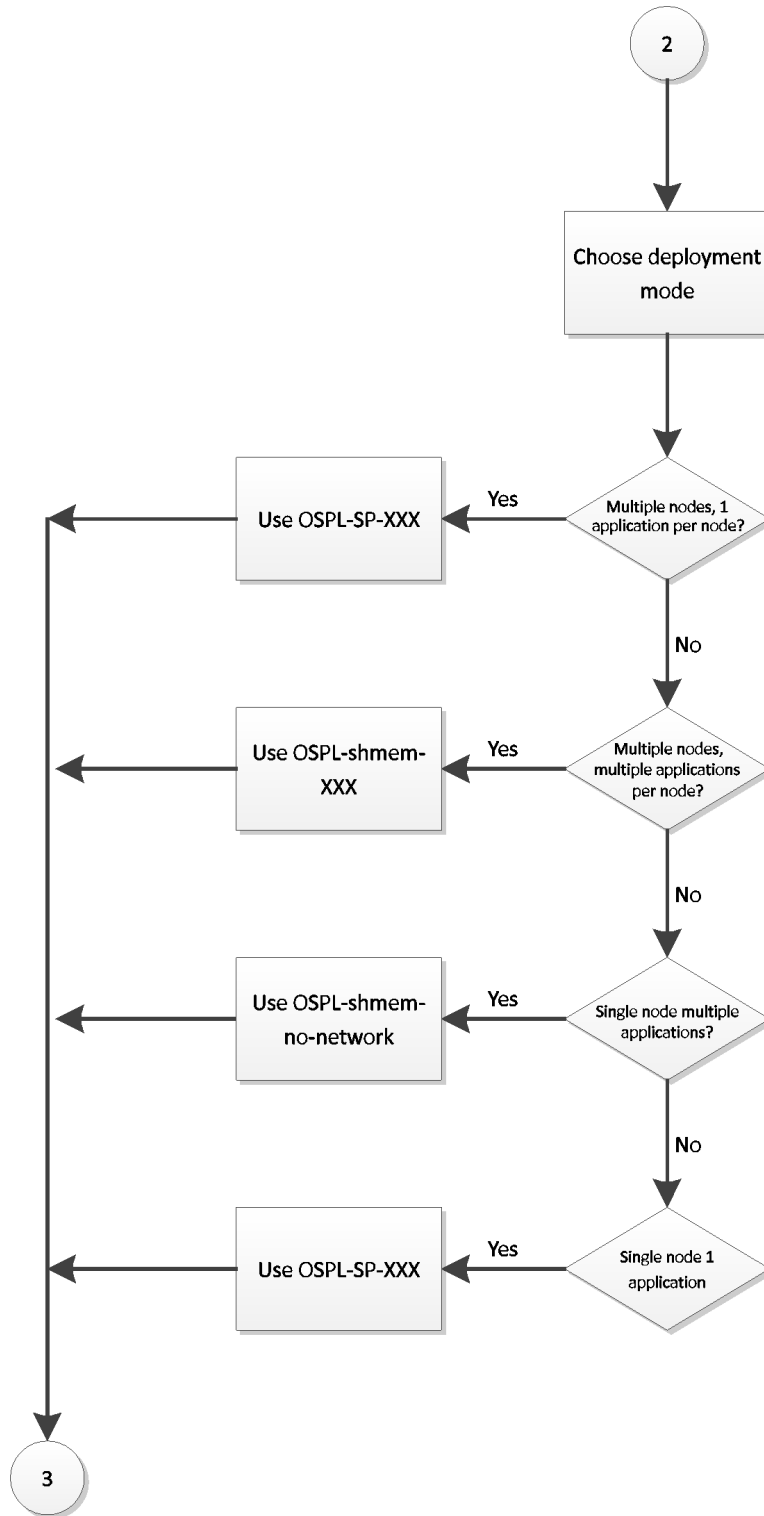


Fig. 2: Selecting the architectural deployment mode

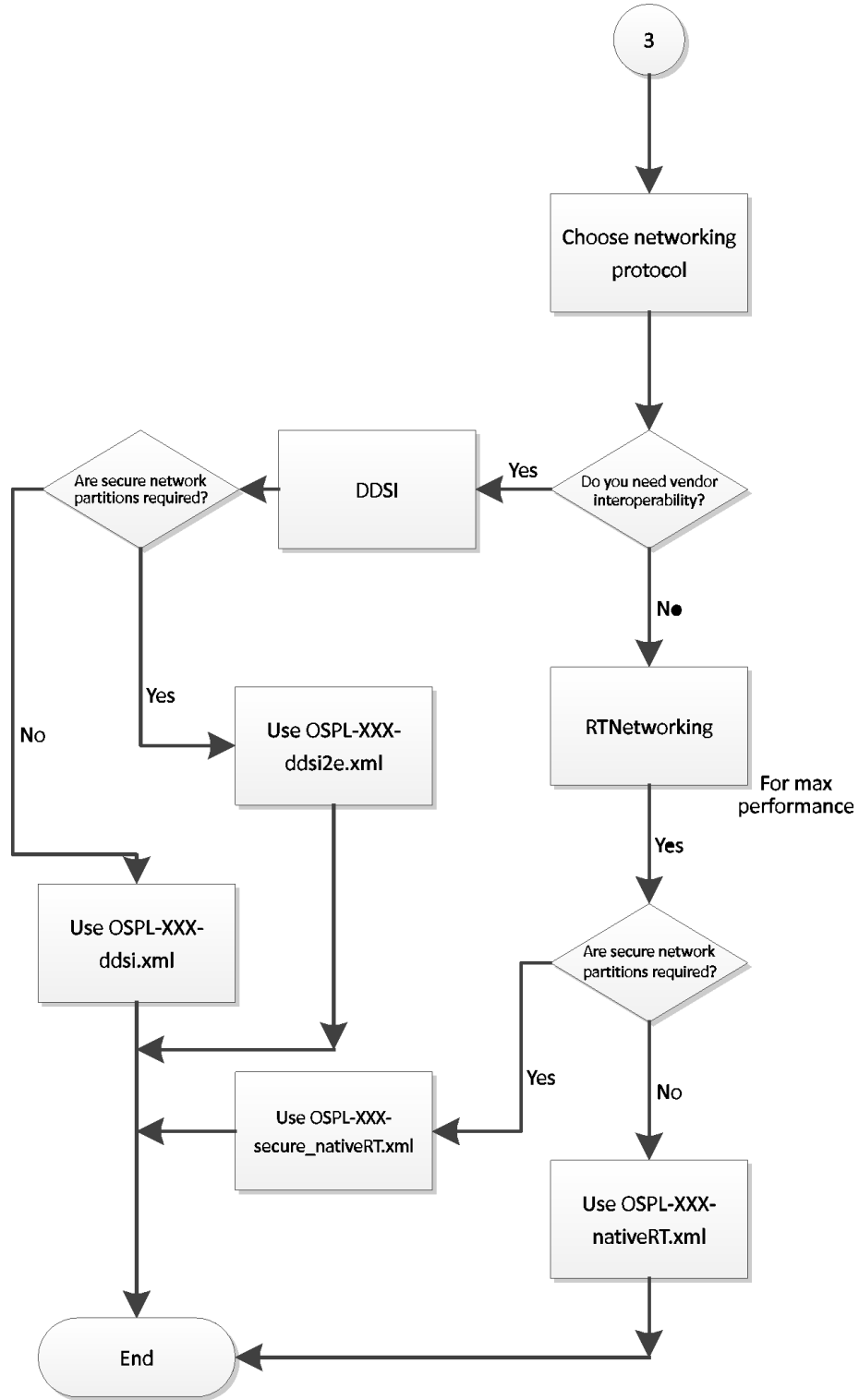


Fig. 3: Selecting the network protocol options

## HOW TO RUN OPENSPLICE

### 7.1 The OpenSplice Environment

A release file is provided with the OpenSplice installation which contains the environment variables that are required. Create an OpenSplice environment as follows.

**First:**

**Linux**

Open a shell and source the `release.com` file from the OpenSplice installation directory.

**Windows**

Open a *Windows Command prompt* and run the `release.bat` file in the OpenSplice installation directory.

**Windows**

Alternatively, use the *OpenSplice Command Prompt* that can be accessed from the Windows *Start* menu (this will implicitly run `release.bat`).



Note that the OpenSplice Launcher tool also provides a `Console` option which sets up the OpenSplice environment.

**Next:**

Set the `OSPL_URI` variable to refer to the OpenSplice configuration that is required (see the section *How to select the Architectural Mode*).

### 7.2 Running Single Process and Shared Memory Modes

- With an `OSPL_URI` variable referring to a *Single Process* deployment, you just need to start the DDS application process. The `create_participant()` operation, which is the entry into the DDS Domain, will create the entire DDS infrastructure within the application process and the services will be started as threads.
- With an `OSPL_URI` variable referring to a *Shared Memory* deployment, it is necessary to start the DDS infrastructure before starting your DDS application processes. That is done by using the `ospl` utility tool:

```
ospl start
# now run the DDS application processes as normal
ospl stop
```

## PERFORMANCE TESTS AND EXAMPLES

To make the evaluation process as easy as possible, OpenSplice is shipped with dedicated performance tests that can be used to measure latency and throughput. The tests are simple and clear, allowing the user to obtain performance results easily.

The easiest way to build and run the performance tests is to use the OpenSplice *Launcher* tool. In the *Examples* menu select the specific example and the appropriate language and configuration. Click the *Compile Example* button and then *Run Example*. This will run the DDS applications, and if running with a shared memory configuration it will also manage the starting and stopping of OpenSplice.

OpenSplice also provides dedicated performance testing scripts which:

- Test multiple API bindings
- Use a varying range of payload sizes
- Timestamp and append results to a CSV file
- Set process priority and CPU affinity

Please see the `html` files for the individual performance tests for details of how to run these scripts.

### 8.1 Round-Trip Latency Performance

The latency of a DDS implementation is an expression of how fast data can be delivered between two DDS applications. *Round-trip latency* is the time taken for an individual DDS data sample to be delivered from Application A to Application B and back again, so importantly it includes metrics for both data delivery and reception.

The easiest way to build and run the performance tests is to use the OpenSplice *Launcher* tool as explained above.

Alternatively, to manually build and run the round-trip performance test, for example for the ISO C++ API:

#### Linux

```
# In an OpenSplice environment:
cd $OSPL_HOME/examples/dcps/RoundTrip/isocpp
make

cd $OSPL_HOME/examples/dcps/RoundTrip/isocpp
# If using shared memory do "ospl start"
./pong
# If using shared memory do "ospl stop"

# In another OpenSplice environment:
```

(continues on next page)



(continued from previous page)

```
cd $OSPL_HOME/examples/dcps/RoundTrip/isocpp
# If using shared memory do "ospl start"
./ping 20 100
# If using shared memory do "ospl stop"
```

**Windows**

```
# Load the OpenSplice examples project solution
# into Visual Studio and build the required projects

# In an OpenSplice environment:
cd %OSPL_HOME%\examples\dcps\RoundTrip\isocpp
# If using shared memory do "ospl start"
pong.exe
# If using shared memory do "ospl stop"

# In another OpenSplice environment:
cd %OSPL_HOME%\examples\dcps\RoundTrip\isocpp
# If using shared memory do "ospl start"
ping.exe 20 100
# If using shared memory do "ospl stop"
```

The ping application will report the roundtrip time taken to send DDS data samples back and forth between the applications. The test utilizes the ReliabilityQoS set to RELIABLE by default in order to show the maximal performance whilst maintaining the guaranteed delivery of DDS samples. See the README file for the test for further details.

**The lowest roundtrip latency may be achieved by tuning the test parameters appropriately.**

As mentioned above, the performance testing script described in the `html` for the example is a convenient way to test and record the running of this test.



Note that the default OSPL\_URI value refers to a *Single Process* deployment with *DDSI* networking.

- To observe the best performance within a node it is suggested that you use a *Shared Memory* configuration.
- To observe the best performance between nodes it is suggested that you use an *RTNetworking* service configuration.

## 8.2 Throughput Performance

The throughput of a DDS implementation is an expression of the rate of data delivery through the DDS system. Measured in bits per second, it describes the ability of the DDS implementation to effectively deliver DDS data without data loss.

As with the round-trip test, the easiest way to build and run the throughput performance test is to use the OpenSplice *Launcher* tool.

Alternatively, to manually build and run the throughput performance test, for example for the ISO C++ API:

**Linux**

```
# In an OpenSplice environment:
cd $OSPL_HOME/examples/dcps/Throughput/isocpp
```

(continues on next page)

(continued from previous page)

```

make
cd $OSPL_HOME/examples/dcps/Throughput/isocpp
# If using shared memory do "ospl start"
./publisher
# If using shared memory do "ospl stop"

# In another In an OpenSplice environment:
cd $OSPL_HOME/examples/dcps/Throughput/isocpp
# If using shared memory do "ospl start"
./subscriber
# If using shared memory do "ospl stop"

```

### Windows

```

# Load the OpenSplice examples project solution
# into Visual Studio and build the required projects

# In an OpenSplice environment:
cd %OSPL_HOME%\examples\dcps\Throughput\isocpp
# If using shared memory do "ospl start"
publisher.exe
# If using shared memory do "ospl stop"

# In another OpenSplice environment:
cd %OSPL_HOME%\examples\dcps\Throughput\isocpp
# If using shared memory do "ospl start"
subscriber.exe
# If using shared memory do "ospl stop"

```

The subscriber application will report the DDS data throughput by default once per second. This and many other aspects of the test can be configured on the command line. The test utilizes the `ReliabilityQoS` set to `RELIABLE` by default in order to show the maximal performance whilst maintaining the guaranteed delivery of DDS samples. See the README file for the test for further details.

**The maximum throughput may be achieved by tuning the test parameters appropriately.**

As mentioned above, the performance testing script described in the `html` for the example is a convenient way to test and record the running of this test.



Note that the default `OSPL_URI` value refers to a *Single Process* deployment with *DDSI* networking.

- To observe the best performance within a node it is suggested that you use a *Shared Memory* configuration.
- To observe the best performance between nodes it is suggested that you use an *RTNetworking* service configuration.

### 8.2.1 Achieving Maximum Throughput

Where there is a requirement to support continuous flows or ‘streams’ of data with minimal overhead consider the use of OpenSplice Streams. The ability to deliver potentially millions of samples per second is realized by the Streams feature transparently batching (packing and queuing) the periodic samples.

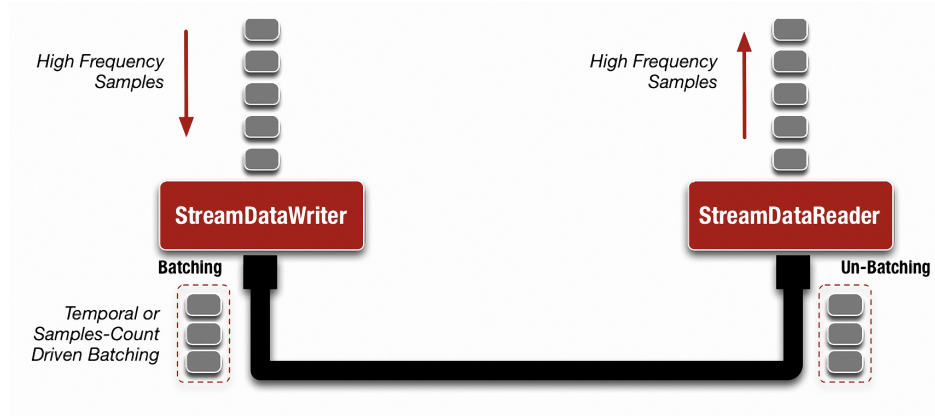


Fig. 1: Streams Architecture

The streams performance example is located in the `examples/streams` directory within the installation.

## BIBLIOGRAPHY

### OMG DDS 1.2

Object Management Group,  
*'Data Distribution Service for Real-Time Systems Version 1.2'*,  
Available specification formal/07-01-01

### OMG DDS 1.4

Object Management Group,  
*'Data Distribution Service for Real-Time Systems Version 1.4'*,  
Available specification formal/15-04-10

### OMG DDSI 2.1

Object Management Group,  
*'The Real-Time Publish-Subscribe Wire Protocol DDS  
Interoperability Wire Protocol Specification Version 2.1'*,  
Document Number: formal/2009-01-05

### OMG DDSI 2.2

Object Management Group,  
*'The Real-Time Publish-Subscribe Wire Protocol DDS  
Interoperability Wire Protocol Specification Version 2.2'*,  
Document Number: formal/2014-09-01

### OMG DDS XTYPES 1.0

Object Management Group,  
*'Extensible and Dynamic Topic Types for DDS Version 1.0'*,  
Document Number: formal/2012-11-10

## CONTACTS & NOTICES

### 10.1 Contacts

**ZettaScale Technology**  
Suite 1, The Honeycomb  
The Watermark  
Gateshead  
Tyne and Wear  
NE11 9SZ  
UK

**ZettaScale Technology**  
Les Algorithmes  
91190 Saint Aubin  
France

Web: <https://www.zettascale.tech>

Contact: <https://www.zettascale.tech>

E-mail: [contact@zettascale.tech](mailto:contact@zettascale.tech)

LinkedIn: <https://www.linkedin.com/company/zettascaletech/>

Twitter: <https://twitter.com/zettascaletech>

### 10.2 Notices

**Copyright** © 2023 Zettascale Technology. All rights reserved.

*This document may be reproduced in whole but not in part. The information contained in this document is subject to change without notice and is made available in good faith without liability on the part of ZettaScale Technology. All trademarks acknowledged.*